perplexity

Step 3: How to perform regression testing

The practical workflow is: understand what changed, choose the right tests, run them in a controlled environment, investigate failures, then maintain the test suite. This prevents regression testing from becoming a slow, unreliable checklist. [1] [2]

1. Identify what changed

Start with **change impact analysis**: review the pull request, commit messages, bug ticket, or feature specification to determine which modules, APIs, screens, and user flows were touched. Also look for downstream dependencies—for example, a change to login may affect user profiles, orders, or payments. [3] [1]

Ask:

- Which feature was changed?
- Which other features depend on it?
- What is the business impact if it breaks?
- Are there new behaviors that need new test cases? [4] [1]

2. Select the tests

Choose tests that directly cover the changed area plus related critical flows. Teams commonly prioritize by risk, business value, and past failure history rather than mechanically rerunning every test. [2] [3]

Situation	Good test selection
Small change to the search bar	Search tests plus related navigation and results tests
Payment bug fix	Payment tests, order confirmation, refund, and checkout tests
Database or API update	API, data validation, integration, and affected user-flow tests
Major release	Full regression suite plus critical smoke tests [3] [4]

3. Prepare the environment and data

Run tests in a dedicated test environment that closely matches production configuration, including database versions, service endpoints, roles, and settings. Use predictable test data and keep it separate from production data, because inconsistent data can create false failures. [5] [4]

Good practices:

- Reset or create the data each test needs.
- Keep tests independent so one failed test does not cause many others to fail.
- Capture evidence automatically: screenshots, logs, network responses, and the data used. [6] [7]

4. Execute the tests

Run the selected suite manually or automatically. In a CI/CD setup, a fast smoke or targeted regression suite may run on every pull request, while a fuller suite may run nightly or before release. [7] [8]

For each test, record:

- Test name and version tested
- Pass or fail status
- Actual result
- Logs, screenshots, and reproduction steps for failures
- Whether the failure is reproducible [5] [6]

5. Analyze failures and report defects

A failed test does not automatically mean the application is broken. First separate a genuine regression from a **flaky test**—one that passes and fails inconsistently—or an environment/data problem. [3] [4]

For a real defect, report:

- Clear title and severity
- Steps to reproduce
- Expected versus actual behavior
- Screenshots or logs
- Affected test case and likely feature area [6] [3]

6. Fix, retest, and maintain the suite

After a developer fixes the defect, rerun the failed test and relevant regression tests to ensure the fix worked and did not create another problem. Then update the suite: add tests for new behavior or fixed bugs, revise outdated expectations, and remove tests for removed features. [2] [4] [3]

A useful rule is to add a regression test for every production bug fix, so the same issue is less likely to return silently. [4]

Beginner example

Suppose a team changes the **Add to Cart** button:

1. **Identify impact:** Cart page, cart count, checkout, and inventory display may be affected.
2. **Select tests:** Add item, remove item, update quantity, view cart, proceed to checkout, and complete payment.
3. **Prepare:** Use a test account, sample products, and a clean test database.
4. **Execute:** Run automated tests for cart and checkout; manually check the visual layout.
5. **Analyze:** The checkout test fails because the cart total is incorrect.
6. **Fix and maintain:** Developer fixes the total calculation; team reruns checkout tests and adds a test for the discount-plus-cart-total scenario. [2] [3]

Step 4 preview

Next, I can explain **Step 4: Types and strategies of regression testing**, including:

- Corrective, selective, progressive, and complete regression testing
- Retesting versus regression testing
- How teams decide between smoke, targeted, and full regression runs
- A simple beginner strategy for building your first suite

Would you like to continue to **Step 4**?

⁂

1. https://swehb.nasa.gov/spaces/SWEHBVD/pages/102695526/SWE-191+-+Software+Regression+Testing
2. https://dl.acm.org/doi/pdf/10.1145/308769.308790
3. https://www.testrail.com/blog/regression-testing/
4. https://www.flagsmith.com/blog/automated-regression-testing
5. https://www.minitap.ai/magazine/complete-regression-testing-guide
6. https://www.virtuosoqa.com/post/regression-testing-best-practices
7. https://katalon.com/resources-center/blog/automated-regression-testing-guide
8. https://circleci.com/blog/regression-testing-and-how-to-automate-it-with-ci/
9. https://www.katalon.com/resources-center/blog/how-to-maintain-regression-tests
10. https://www.augmentcode.com/learn/regression-testing-defined-purpose-types-and-best-practices
11. https://www.zoho.com/qengine/know/regression-testing.html
12. https://www.ibm.com/think/topics/regression-testing
13. https://www.virtuosoqa.com/post/basics-of-regression-testing
14. https://www.flagsmith.com/blog/regression-testing
15. https://www.browserstack.com/guide/regression-testing

16. https://www.tesbo.io/insights/regression-testing-explained-types-process-and-how-to-pick-a-smaller-suite-that-still-catches-bugs